RAJALAKSHMI ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

Department of Cyber Security

2026–2027 – Odd Semester

III Year / V Semester

CS23533 – FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Cyber Intelligence System
Project Review – 2



Rajalakshmi Engineering College

Department of Cyber Security

Project Overview & Implementation

- SHERLOCK is a web-based, explainable cybersecurity expert system built around Classical AI.
- The prototype accepts a user-described cybersecurity situation through a simple interface.
- A Python rule base represents known threat patterns using explicit IF–THEN conditions.
- The inference workflow connects matched facts to a threat category, severity and explanation.
- The current review focuses on implementation structure, output design and the remaining evaluation work.

Implementation Requirements

- The system must accept plain-language descriptions of observable cybersecurity events.
- Inputs should be assessed only against facts and rules represented in the knowledge base.
- The result should distinguish a supported threat match from an uncertain or suspicious situation.
- Each result should show the reason for the classification and practical defensive guidance.
- The prototype must avoid claiming that rule-based analysis guarantees detection of every attack.

Motivation

- Make cybersecurity situations easier for non-expert users to interpret.
- Apply FOAI Classical AI concepts to a practical cybersecurity problem.
- Use explicit knowledge, facts and IF–THEN rules for transparent reasoning.
- Explain what matched, why it matched and what action should be considered.
- Support security awareness through severity, recommendations and applicable MITRE ATT&CK references.

Updated Implementation Objectives

- Organize cybersecurity knowledge as explicit facts and IF–THEN rules.
- Connect the Flask interface to the threat-analysis function and its returned result.
- Present the threat name, category, severity, evidence and recommended action clearly.
- Keep the inference process explainable and traceable to matched rules.
- Prepare representative test cases for known, ambiguous and unmatched situations.

Literature Survey

S.No.	Author & Year	Method / Technique	Key Findings	Limitations
1	Buchanan & Shortliffe, 1984	Rule-based expert system	MYCIN demonstrated explicit knowledge representation and inference rules for expert decision support.	Knowledge is limited to the encoded domain rules.
2	Giarratano & Riley, 2005	Expert systems; knowledge representation; inference	Expert systems combine a knowledge base with an inference engine to derive decisions.	Performance depends on knowledge quality and coverage.
3	MITRE ATT&CK	Adversary tactics and techniques knowledge base	Provides structured cybersecurity techniques that can support threat classification and mapping.	Mappings must be selected carefully for the observed behaviour.

Existing System

Existing approach

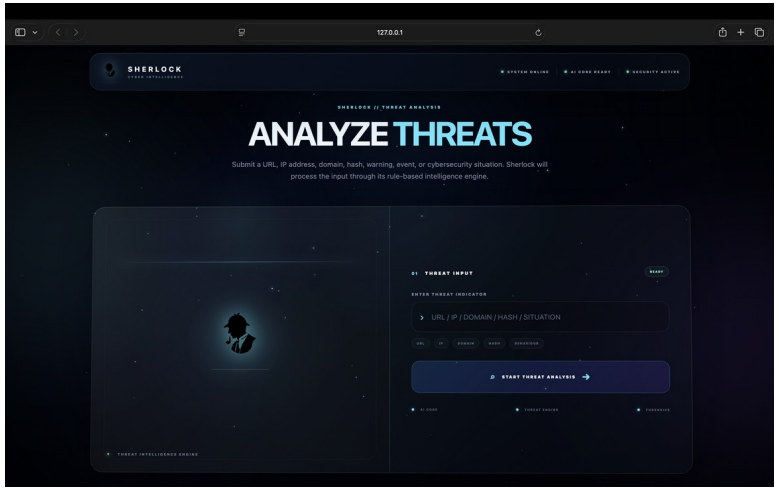
- Manual interpretation, static security guidance and generic alerts.
- Users often need cybersecurity knowledge to interpret the event.

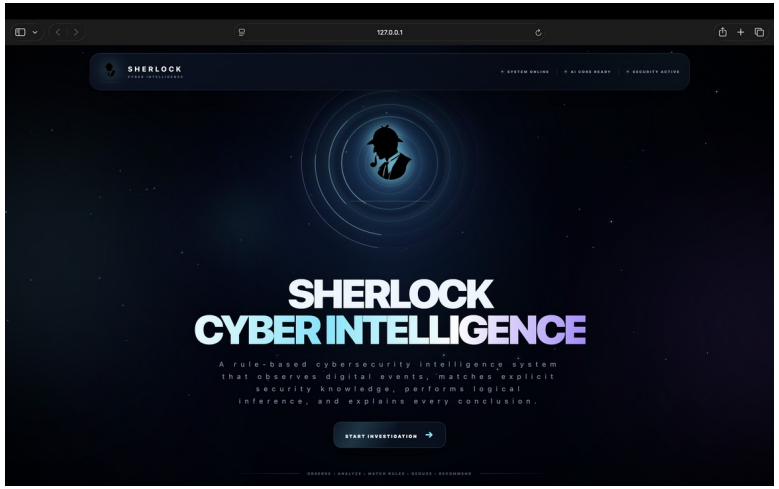
Advantages

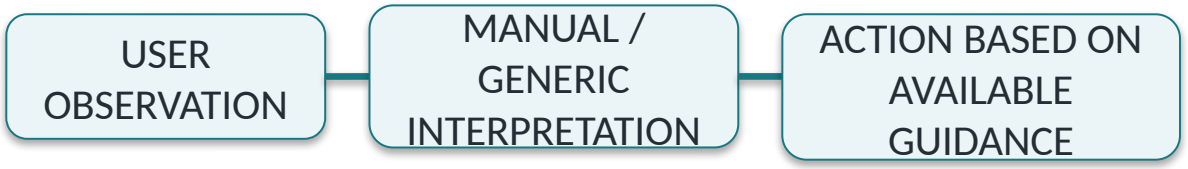
- Security information and known attack patterns are available.

Limitations

- Generic alerts may not explain matched evidence or next action.
- Fixed labels may not adequately describe ambiguous situations.

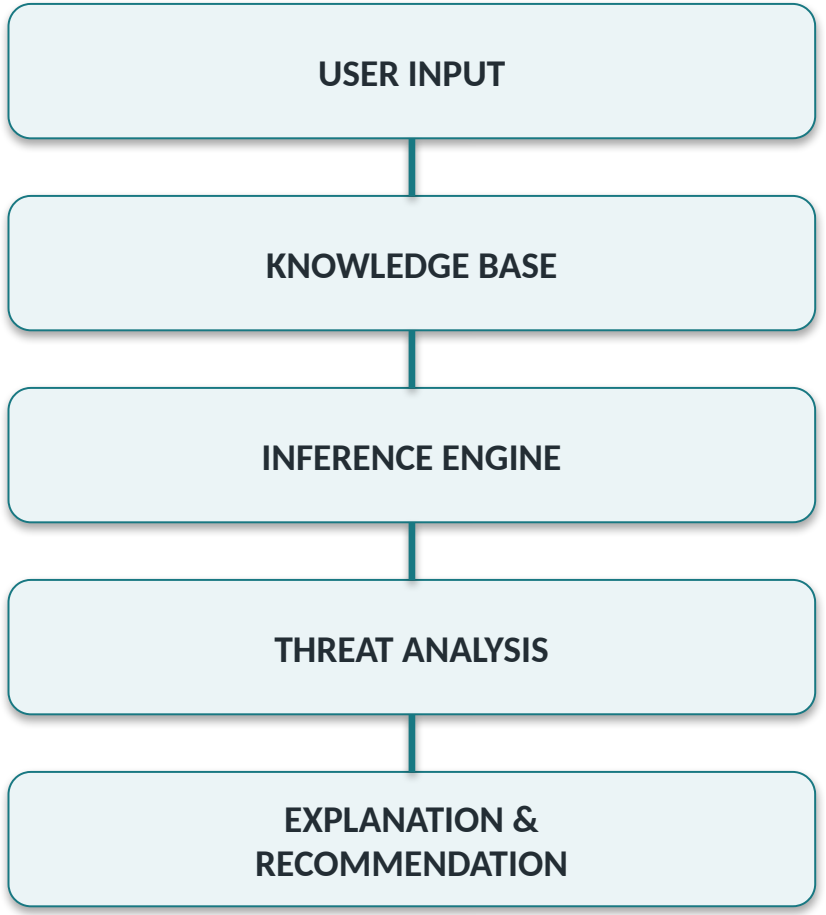




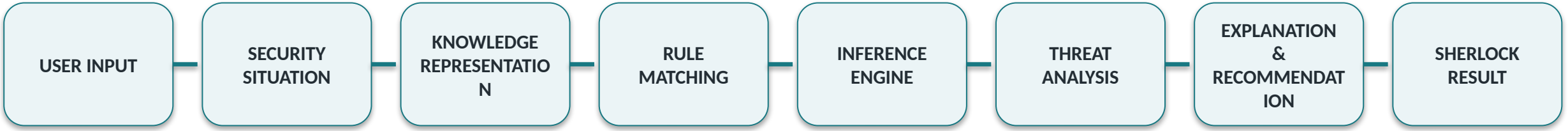


Proposed System – Implementation View

- The web interface collects a user-described cybersecurity situation.
- The backend passes the input to the rule-based analysis engine.
- The engine checks observable indicators against the encoded knowledge base.
- The result view presents a possible threat, severity, matched evidence and recommendations.
- When evidence is insufficient, the output should remain cautious and use a suspicious / potential-threat category.



System Architecture / Block Diagram



Core pipeline: USER INPUT → KNOWLEDGE BASE → INFERENCE ENGINE → THREAT ANALYSIS → EXPLANATION & RECOMMENDATION

Implementation Methodology

1. USER INPUT
Accept a described cybersecurity event.

2. REQUEST HANDLING
Receive the form submission in Flask.

3. RULE EVALUATION
Check indicators against the knowledge base.

4. INFERENCE
Derive a classification from matched rules.

5. RESULT FORMATTING
Prepare severity, evidence and explanation.

6. RESPONSE
Show analysis and defensive guidance.

Implementation Technologies

Programming Language: Python

Web Framework: Flask
Frontend: HTML, CSS, JavaScript
Rule Representation: Explicit IF–THEN conditions
Development Environment: Visual Studio Code
AI Approach: Classical AI expert system
Reasoning Focus: Rule matching and forward inference; backward chaining as a concept
Hardware: No special hardware required

Implementation Modules

Module 1 – User Interface
Collects the cybersecurity situation from the user.

Module 2 – Request Handling
Receives the form submission through the Flask application.

Module 3 – Knowledge Base
Stores the known indicators and IF–THEN rules.

Module 4 – Inference & Analysis
Evaluates rule matches and derives a possible threat and severity.

Module 5 – Explanation & Recommendation
Presents matched evidence, reasoning and suggested defensive action.

Illustrative Analysis Output

USER INPUT

“My account has received hundreds of failed login attempts in a short period.”

SHERLOCK ANALYSIS

Possible threat: Brute Force Attack
Category: Credential Access
Illustrative severity: HIGH

WHY?

Repeated failed authentication attempts are the indicator represented by the corresponding rule. This is an illustrative example, not a measured test result.

RECOMMENDATION

Review authentication logs • Enable MFA • Investigate source addresses • Reset credentials if compromise is suspected

MITRE ATT&CK: T1110 (Brute Force)

Validation note: confirm the output against the implemented rules and record results during systematic testing.

Project Schedule

Activity	Phase 1	Phase 2	Phase 3	Phase 4
Literature & problem study	✓	✓		
System and rule design	✓	✓	✓	
Prototype implementation		✓	✓	✓
Systematic testing & evaluation				Planned
Final documentation				Planned

Development Status & Evaluation Plan

- ✓ Problem definition and literature survey documented
- ✓ Classical AI approach and rule-based architecture specified
- ✓ Knowledge-base and inference workflow designed
- ✓ Web-based prototype reported as implemented in Review 1
- → Next: verify each rule with controlled input scenarios
- → Next: record correct classifications, missed matches and false matches
- → Next: check severity, evidence and recommendation consistency
- ☐ Systematic test results and final evaluation remain to be documented

Remaining Work

- Expand and review the coverage of cybersecurity rules.
- Test common scenarios such as repeated failed logins, suspicious links and unknown software.
- Include ambiguous inputs and cases where no rule should match.
- Check whether the displayed evidence supports the assigned category and severity.
- Review overlapping rule matches and define predictable conflict handling.
- Verify MITRE ATT&CK references before presenting them as applicable mappings.
- Complete test documentation, limitations and final demonstration.

Conclusion

- SHERLOCK applies Classical AI expert-system ideas to cybersecurity decision support.
- Its design combines a knowledge base, explicit rules and an inference workflow.
- The interface is intended to make the reasoning and recommended actions understandable.
- Reliable evaluation requires representative test cases and recorded outcomes.
- The next milestone is systematic validation and refinement of the rule base.
- SHERLOCK is a decision-support prototype, not a replacement for security tools or expert investigation.

References

[1] B. G. Buchanan and E. H. Shortliffe, Rule-Based Expert Systems: The MYCIN Experiments, Addison-Wesley, 1984.

[2] J. Giarratano and G. Riley, Expert Systems: Principles and Programming, 4th ed., Course Technology, 2005.

[3] MITRE, MITRE ATT&CK Enterprise knowledge base and technique references.

[4] CS23533 – Foundations of Artificial Intelligence, Rajalakshmi Engineering College, course syllabus and concepts.

THANK YOU

Questions & Suggestions